Supply True Signatures to vmlinux BTF

Yonghong Song and Alan Maguire

Current vmlinux BTF - Source Signature

- The function signature is based on source code.

```
typedef struct {
        union {
                void            *kernel;
                void __user     *user;
        };
        bool            is_kernel : 1;
} sockptr_t;

typedef sockptr_t bpfptr_t;
static int map_create(union bpf_attr *attr, bpfptr_t uattr) { … }
```

Current vmlinux BTF - Fully Inlined Functions

- The fully inlined function does not have an entry in kallsyms or vmlinux BTF.

```
static bool btf_type_is_datasec(const struct btf_type *t)
{
        return BTF_INFO_KIND(t->info) == BTF_KIND_DATASEC;
}
$ grep btf_type_is_datasec log.vmlinux.btf
$
$ grep btf_type_is_datasec System.map
$
```

- pahole does capture/process all functions in kallsyms.
- vmlinux BTF may have less functions since pahole may filter out some functions except e.g., more than one same-name static functions, etc.

Compiler Optimization May Change Signatures (LLVM)

Source Level:

```
typedef struct {
        union {
                void            *kernel;
                void __user     *user;
        };
        bool            is_kernel : 1;
} sockptr_t;
typedef sockptr_t bpfptr_t;
static int map_create(union bpf_attr *attr,
                      bpfptr_t uattr) { … }
```

After Optimization:

// LLVM IR

define internal fastcc i32 @map_create(ptr noundef %0, i8 range(i8 0, 2) %1) { … }

Basically, the signature actually becomes

static int map_create(union bpf_attr *attr, bool is_kernel) { … }

Signature Change between GCC and LLVM

- LLVM maintains the same function name even when signature changed.
- GCC has various suffixes for function name to indicate a signature change and why.

```
mode_strip_umask.isra.0
sysctl_print_dir.part.0.isra.0
find_newest_super_backup.isra.0
rcu_lock_acquire.constprop.0
kzalloc_noprof.constprop.0
uncore_pci_exit.part.0
dump_blkd_tasks.part.0.constprop.0
…
```

Get True Signatures in LLVM

```
define internal fastcc i32 @map_create(ptr noundef %0, i8 range(i8 0, 2) %1) unnamed_addr #0 align 16 !dbg !14902 {
  #dbg_value(ptr poison, !14907, !DIExpression(DW_OP_LLVM_fragment, 0, 64), !23117)
  #dbg_value(i8 %1, !14907, !DIExpression(DW_OP_LLVM_fragment, 64, 8), !23117)
  #dbg_value(ptr %0, !14906, !DIExpression(), !23117)
  …
 %3 = getelementptr i8, ptr %0, i64 16, !dbg !23120
 ……
```

!14907 and !14906 refers to llvm debuginfo.

The argument %0 has information in !14906

The argument %1 has information in !14907.

!DIExpression(DW_OP_LLVM_fragment, 64, 8) with !14907 implies to take the value with offset 64bit and size 8bit for !14907.

Get True Signatures in LLVM

!14906 = !DILocalVariable(name: "**attr**", arg: 1, scope: !14902, file: !14315, line: 1375, type: !643)

!14907 = !DILocalVariable(name: "**uattr**", arg: 2, scope: !14902, file: !14315, line: 1375, type: !13952)
!13952 = !DIDerivedType(tag: DW_TAG_typedef, name: "bpfptr_t", file: !13953, line: 9, baseType: !2377)
!2377 = !DIDerivedType(tag: DW_TAG_typedef, name: "sockptr_t", file: !2378, line: 20, baseType: !2379)
!2379 = distinct !DICompositeType(tag: DW_TAG_structure_type, file: !2378, line: 14, size: 128, elements: !2380)
!2380 = !{!2381, !2389}
⇒
!2381 = !DIDerivedType(tag: DW_TAG_member, scope: !2379, file: !2378, line: 15, baseType: !2382, size: 64) (**union)**

⇒
!2389 = !DIDerivedType(tag: DW_TAG_member, name: "**is_kernel**", scope: !2379, file: !2378, line: 19, baseType: !615, **size: 1, offset: 64**, flags: DIFlagBitField, extraData: i64 64) (**bool**)
!615 = !DIDerivedType(tag: DW_TAG_typedef, name: "bool", file: !616, line: 35, baseType: !617)
!617 = !DIBasicType(name: "_Bool", size: 8, encoding: DW_ATE_boolean)

Summary To Get True Signatures in LLVM

- Utilize llvm dbg_{value, declare, assign} to get the semantic and name of the parameter.
- The return value either becomes ‘void’ after the optimization or maintains the original return type.
- Details in link: https://github.com/llvm/llvm-project/pull/165310

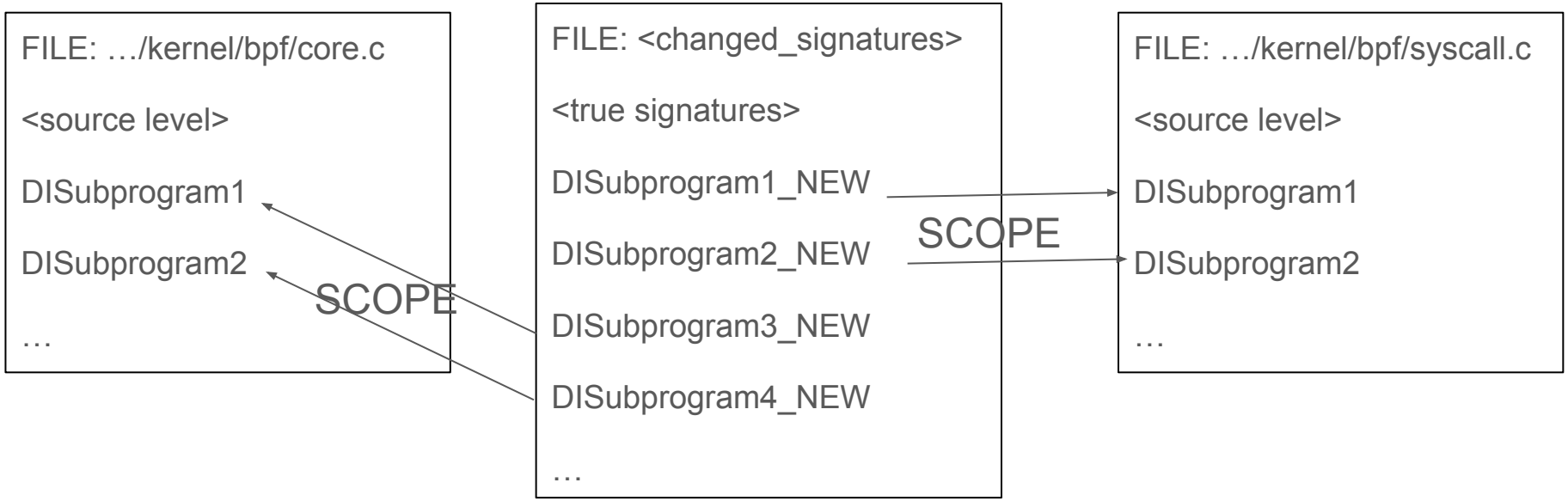
Remarks in LLVM for True Signatures

- It is possible that additional or different dbg patterns may appear in LLVM. In such cases, true signatures may not be available and remarks can be used to find which function and why needs to be improved.
- To enable remarks, add the following compiler flags:
  - non-LTO: -fsave-optimization-record -foptimization-record-passes=emit-changed-func-debuginfo
  - LTO: --opt-remarks-filename <prefix> --opt-remarks-passes emit-changed-func-debuginfo
- The following is a remark example:

```
--- !Passed
Pass:            emit-changed-func-debuginfo
Name:            FindNoDIVariable
DebugLoc:        { File: test.c, Line: 1, Column:
0 }
Function:        callee
Args:
  - String:          'create a new int type '
  - ArgName:           ''
  - String:          '('
  - ArgIndex:        '0'
  - String:          ')'
```

For current bpf-next, build with the above additional flags, there are no remarks with command line ***make LLVM=1 V=1 -j***.

Encode True Signature to IR DebugInfo



In final dwarf generation, DISubprogram{1,2}_NEW will be in file syscall.c and DISUbprogram{3,4}_NEW will be in file core.c.

Dwarf Generation

- There are no dwarf specification about how to encode new signatures. Also we want to avoid changes for lldb debugger. So the new signature is encoded as

```
DW_TAG_compile_unit
  DW_TAG_subprogram
    …
  DW_TAG_inlined_subroutine
    … // true function signature
  DW_TAG_inlined_subroutine
    … // true function signature
```

This should have minimum impact for dwarf-related tools or debugger.

Dwarf Encoding Example for map_create()

```
0x0187d21c:   DW_TAG_inlined_subroutine
              DW_AT_name      ("map_create")
              DW_AT_type      (0x0184b340 "int")
              DW_AT_artificial        (true)
              DW_AT_specification     (0x01868060 "map_create")

0x0187d228:     DW_TAG_formal_parameter
                DW_AT_name    ("attr")
                DW_AT_type    (0x0184b52e "bpf_attr *")

0x0187d230:     DW_TAG_formal_parameter
                DW_AT_name    ("is_kernel")
                DW_AT_type    (0x0184ccd2 "bool")

0x0187d238:     NULL
```

DW_AT_artificial implies this function is generated by the compiler.

DW_AT_specification points to the original subprogram.

Dwarf Encoding Example for <func>.llvm.<hash>() with LTO

```
0x0060dd56:   DW_TAG_inlined_subroutine
                DW_AT_name      ("enable_step.llvm.5694997634595845 54")
                DW_AT_artificial        (true)
                DW_AT_specification     (0x0060d396 "enable_step")

0x0060dd5d:     DW_TAG_formal_parameter
                  DW_AT_name    ("child")
                  DW_AT_type    (0x006046b9 "task_struct *")

0x0060dd64:     DW_TAG_formal_parameter
                  DW_AT_name    ("block")
                  DW_AT_type    (0x00605b99 "bool")

0x0060dd6b:     NULL
```

In kallsyms, enable_step() function does not exist, the actual function is enable_step.llvm.<hash>() and it has the same source level signature.

Summary of New Functions Into Pahole

- The function without ‘.’ : the function signature has changed due to compiler transformation.
- The function with ‘.’ : the function signature may or may not be the source level signature.
  - Note that in LLVM at LTO mode, a static function foo() can be promoted to foo.llvm.<hash>() due to cross-file inlining. In such cases, the original function foo() may disappear in symbol table.
- All new functions exist in kallsyms.

Handle New Function like map_create() (Pahole)

- There will be two functions having name map_create() in pahole. One has source level signature and the other has true signature.
- Pahole will use the true signature to encode for vmlinux BTF.
- See https://lore.kernel.org/bpf/20251130040340.2636458-1-yonghong.song@linux.dev/ for details.

In vmlinux BTF:

```
static int map_create(union bpf_attr *attr,   bool is_kernel) { … }
```

Handle New Function With Dot

```
0x0060dd56:   DW_TAG_inlined_subroutine
                DW_AT_name      ("enable_step.llvm.7352441459262763370")
                DW_AT_artificial        (true)
                DW_AT_specification     (0x0060d396 "enable_step")
…

[root@arch-fb-vm1 bpf-next]# grep enable_step /proc/kallsyms
ffffffff81285fc0 T __pfx_enable_step.llvm.7352441459262763370
ffffffff81285fd0 t enable_step.llvm.7352441459262763370
[root@arch-fb-vm1 bpf-next]# grep enable_step /sys/kernel/tracing/available_filter_functions
Enable_step.llvm.7352441459262763370
[root@arch-fb-vm1 bpf-next]# grep enable_step /sys/kernel/tracing/available_filter_functions_addrs
ffffffff81285fd4 enable_step.llvm.7352441459262763370
```

Handle New Function With Dot

- So the idea is to have vmlinux BTF func name matches kallsyms.
- In pahole this implies that the function suffix (e.g., .llvm.<hash>) will be preserved when checking ksym functions.

In vmlinux BTF:

```
static void enable_step.llvm.7352441459262763370(struct task_struct *child, bool block) { … }
```

New Feature in Pahole

- A new feature ‘true_signature’ is implemented in pahole to control whether true signatures should be encoded in vmlinux BTF or not.
- Currently we skip functions with inconsistent representations, but true_signature support would override that.

Kernel Build

- Currently, the true signature support is off by default in LLVM. So to enable true signature support, the kernel build system needs the following:

```
Thin-lto mode:
--- a/Makefile
+++ b/Makefile
@@ -1002,6 +1002,11 @@ endif
 ifdef CONFIG_LTO_CLANG
 ifdef CONFIG_LTO_CLANG_THIN
 CC_FLAGS_LTO   := -flto=thin -fsplit-lto-unit
+ifeq ($(call test-ge, $(CONFIG_PAHOLE_VERSION), 131),y)
+ifeq ($(call clang-min-version, 220000),y)
+KBUILD_LDFLAGS  += $(call ld-option, -mllvm -enable-changed-func-dbinfo)
+endif
+endif
 else
 CC_FLAGS_LTO   := -flto
 endif
```

Kernel Build

None-lto mode:

```
--- a/Makefile
+++ b/Makefile
…
@@ -1009,6 +1014,12 @@ CC_FLAGS_LTO     += -fvisibility=hidden

 # Limit inlining across translation units to reduce binary size
 KBUILD_LDFLAGS += -mllvm -import-instr-limit=5
+else
+ifeq ($(call test-ge, $(CONFIG_PAHOLE_VERSION), 131),y)
+ifeq ($(call clang-min-version, 220000),y)
+KBUILD_CFLAGS  += -mllvm -enable-changed-func-dbinfo
+endif
+endif
 endif

 ifdef CONFIG_LTO
```

Kernel Build

Enable true_signature support in pahole:

```
-diff --git a/scripts/Makefile.btf b/scripts/Makefile.btf
index db76335dd917..58cef0f37c4c 100644
--- a/scripts/Makefile.btf
+++ b/scripts/Makefile.btf
@@ -24,6 +24,7 @@ else
 pahole-flags-$(call test-ge, $(pahole-ver), 126)  = -j$(JOBS)
--btf_features=encode_force,var,float,enum64,decl_tag,type_tag,optimized_func,consistent_func,decl_tag_kfuncs

 pahole-flags-$(call test-ge, $(pahole-ver), 130) += --btf_features=attributes
+pahole-flags-$(call test-ge, $(pahole-ver), 131) += --btf_features=true_signature

 ifneq ($(KBUILD_EXTMOD),)
 module-pahole-flags-$(call test-ge, $(pahole-ver), 128) += --btf_features=distilled_base
```

Statistics: Compare no-lto and lto Functions

- without -mllvm -enable-changed-func-dbinfo:
  - no-lto: 66051 functions
  - lto: 66227 functions
- additional functions with -mllvm -enable-changed-func-dbinfo:
  - no-lto: 894 new DW_TAG_inlined_subroutine entries
  - lto: 2991 ...

Statistics: vmlinux BTF size

- no-lto without new signatures: 0x623746
- no-lto with new signatures: 0x62712e
- BTF increase by 14824 bytes

- lto without new signatures: 0x6335b9
- lto with new signatures: 0x642ea3
- BTF increase by 63722 bytes

So looks like the BTF increase should not be a concern here.

Tracing <func>.llvm.<hash> functions

- Previously, tracing function like <func>.llvm.<hash> with fentry won't work since vmlinux BTF does not have a function with name <func>.llvm.<hash>.
- Now it works with some user space handling.
- We could have a helper function in libbpf to find the target name.

```
+       skel = fentry_lto__open();
+       if (!ASSERT_OK_PTR(skel, "fentry_lto__open"))
+               return;
+       /* The fentry target is 'start_thread_common' or 'start_thread_common.llvm.<hash>', search ksyms */
+       target = libbpf_get_full_name_from_ksyms("start_thread_common");
+       prog = skel->progs.test1;
+       bpf_program__set_attach_target(prog, 0, target);
+       err = fentry_lto__load(skel);
+       if (!ASSERT_OK(err, "fentry_lto__load"))
+               goto out;
```

Handle Inlined Functions

- The idea is to allow trace those inlined functions. This can avoid surprise from users when they intend to trace all instances of that function.
- We use DW_TAG_inlined_subroutine which refers to the abstract origin original function
- BTF encoding represents such sites as “locations” with associated address, function prototype and info for retrieving the parameters (where available)
- See below links from Alan
  https://lore.kernel.org/bpf/20251008173512.731801-1-alan.maguire@oracle.com/#r
- https://lore.kernel.org/bpf/20251024073328.370457-1-alan.maguire@oracle.com/

Non-LTO Mode (LLVM)

- The inlining happens before any signature change.
- At InlinerPass, the debuginfo is amended so later dwarf generation can represent the inlining properly.
- The original source signature are used for those inlined functions.
- The final function signature could be different from the source signature if the function is not completely inlined.

… -> Inliner -> DeadArgumentElimination/ArgumentPromotion -> .o files

Example

```
struct t { int a; };
char *tar(struct t *a, struct t *d);
static char * foo(struct t *a, int b, struct t *d)
{
  return tar(a, d);
}
char *bar(struct t *a, struct t *d)
{
  return foo(a, 1, d);
}
```

```
; *** IR Dump After InlinerPass on (bar) ***
define dso_local ptr @bar(ptr noundef %0, ptr noundef %1)
local_unnamed_addr #0 !dbg !14 {
    #dbg_value(ptr %0, !25, !DIExpression(), !27)
    #dbg_value(ptr %1, !26, !DIExpression(), !27)
    #dbg_value(ptr %0, !28, !DIExpression(), !35)
    #dbg_value(i32 1, !33, !DIExpression(), !35)
    #dbg_value(ptr %1, !34, !DIExpression(), !35)
  %3 = call ptr @tar(ptr noundef %0, ptr noundef %1) #2, !dbg !37
  ret ptr %3, !dbg !38
}
…
!28 = !DILocalVariable(name: "a", arg: 1, scope: !29, file: !1, line: 3, type: !19)
!29 = distinct !DISubprogram(name: "foo", scope: !1, file: !1, line: 3, type: !30,
scopeLine: 4, flags: DIFlagPrototyped | DIFlagAllCallsDescribed, spFlags:
DISPFlagLocalToUnit | DISPFlagDefinition | DISPFlagOptimized, unit: !0,
retainedNodes: !32, keyInstructions: true)
!30 = !DISubroutineType(types: !31)
!31 = !{!17, !19, !23, !19}
!32 = !{!28, !33, !34}
!33 = !DILocalVariable(name: "b", arg: 2, scope: !29, file: !1, line: 3, type: !23)
!34 = !DILocalVariable(name: "d", arg: 3, scope: !29, file: !1, line: 3, type: !19)
```

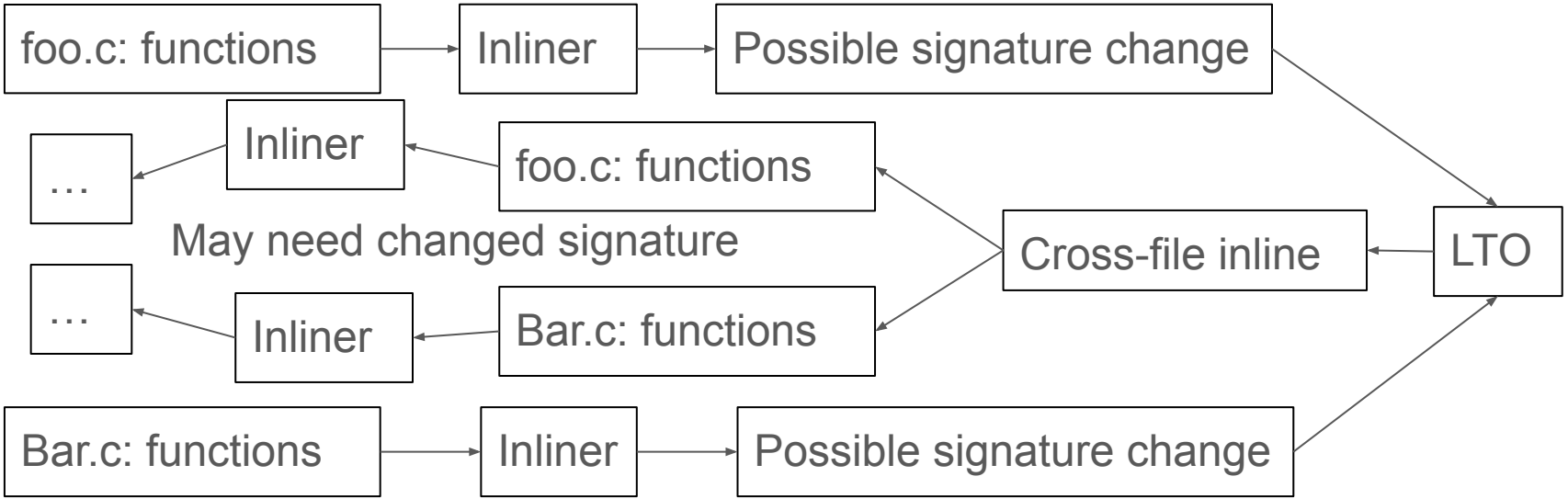
Example

```
0x00000088:     DW_TAG_inlined_subroutine
                  DW_AT_abstract_origin (0x00000023 "foo")
                  DW_AT_low_pc  (0x0000000000000000)
                  DW_AT_high_pc (0x0000000000000005)
                  DW_AT_call_file       ("/home/yhs/tests/inline-example/test.c")
                  DW_AT_call_line       (9)
                  DW_AT_call_column     (10)

0x00000095:       DW_TAG_formal_parameter
                    DW_AT_location      (DW_OP_reg5 RDI)
                    DW_AT_abstract_origin       (0x0000002b "a")
0x0000009c:       DW_TAG_formal_parameter
                    DW_AT_const_value   (1)
                    DW_AT_abstract_origin       (0x00000033 "b")
0x000000a2:       DW_TAG_formal_parameter
                    DW_AT_location      (DW_OP_reg4 RSI)
                    DW_AT_abstract_origin       (0x0000003b "d")
0x000000a9:       NULL
```

Thin-LTO Mode (LLVM)

- At Thin-LTO stage. Cross-file inlining happens, and it is possible that inlined functions need changed signature.



Example

```
struct t { int a; };
char *tar(struct t *a, struct t *d);
static char * foo(struct t *a, int b, struct t *d)
{
  return tar(a, d);
}
char *bar(struct t *a, struct t *d)
{
  return foo(a, 1, d);
}
```

After optimization at non-LTO stage, foo() signature becomes:

char *foo(struct t *a, struct t *d);

```
; *** IR Dump After InlinerPass on (bar) ***
define dso_local ptr @bar(ptr noundef %0, ptr noundef %1)
local_unnamed_addr #0 !dbg !14 {
    #dbg_value(ptr %0, !25, !DIExpression(), !27)
    #dbg_value(ptr %1, !26, !DIExpression(), !27)
    #dbg_value(ptr %0, !28, !DIExpression(), !35)
    #dbg_value(i32 poison, !33, !DIExpression(), !35)
    #dbg_value(ptr %1, !34, !DIExpression(), !35)
  %3 = call ptr @tar(ptr noundef %0, ptr noundef %1) #2, !dbg !37
  ret ptr %3, !dbg !38
}
...
!28 = !DILocalVariable(name: "a", arg: 1, scope: !29, file: !1, line: 3, type: !19)
!29 = distinct !DISubprogram(name: "foo", scope: !1, file: !1, line: 3, type: !30,
scopeLine: 4, flags: DIFlagPrototyped | DIFlagAllCallsDescribed, spFlags:
DISPFlagLocalToUnit | DISPFlagDefinition | DISPFlagOptimized, unit: !0,
retainedNodes: !32, keyInstructions: true)
!30 = !DISubroutineType(cc: DW_CC_nocall, types: !31)
!31 = !{!17, !19, !23, !19}
!32 = !{!28, !33, !34}
!33 = !DILocalVariable(name: "b", arg: 2, scope: !29, file: !1, line: 3, type: !23)
!34 = !DILocalVariable(name: "d", arg: 3, scope: !29, file: !1, line: 3, type: !19)
```

Example

```
0x00000089:     DW_TAG_inlined_subroutine
                DW_AT_abstract_origin (0x00000023 "foo")
                DW_AT_low_pc  (0x0000000000000000)
                DW_AT_high_pc (0x0000000000000005)
                DW_AT_call_file       ("/home/yhs/tests/inline-example/test.c")
                DW_AT_call_line       (9)
                DW_AT_call_column     (10)
0x00000096:       DW_TAG_formal_parameter
                  DW_AT_location      (DW_OP_reg5 RDI)
                  DW_AT_abstract_origin       (0x0000002c "a")
0x0000009d:       DW_TAG_formal_parameter
                  DW_AT_location      (DW_OP_reg4 RSI)
                  DW_AT_abstract_origin       (0x0000003c "d")
0x000000a4:       NULL
```

Caveats

- Inlined functions may have different signatures than the final function signature.
- Some optimized inlined functions may disappear from dwarf.

```
static int callee(int p) { return
p + p * p; }
int caller(void) {
  int x = 100;
  return callee(x);
}
```

No DW_TAG_inlined_subroutine for callee().

```
define dso_local noundef i32 @caller() local_unnamed_addr #0 !dbg !14 {
    #dbg_value(i32 100, !19, !DIExpression(), !20)
  ret i32 10100, !dbg !21
}

…
!14 = distinct !DISubprogram(name: "caller", scope: !1, file: !1, line: 2, type:
!15, scopeLine: 2, flags: DIFlagPrototyped | DIFlagAllCallsDescribed,
spFlags: DISPFlagDefinition | DISPFlagOptimized, unit: !0, retainedNodes:
!18, keyInstructions: true)
!19 = !DILocalVariable(name: "x", scope: !14, file: !1, line: 3, type: !17)
```

For GCC

- GCC provides abstract/concrete relationship where the source DW_AT_subprogram function (abstract, sourcelevel) is referred to from a DW_TAG_inlined_subroutine or DW_TAG_subprogram (concrete) via DW_AT_abstract_origin
- An ordered set of parameters with location info point at their abstract origins too; this allows us to figure out where the parameter is stored. If it is missing it is not in the list or marked as optimized out.
- We get the address of the inline site from the lowpc value.
- This allows us to match a specific ELF function with a specific DWARF representation; important because “foo.isra.0” in ELF is called “foo” in DWARF; name transformations are not reflected there.

For GCC

```
DW_TAG_subprogram/DW_TAG_inline_subroutine
    DW_AT_abstract_origin // Reference to abstract function “foo”.
    DW_AT_low_pc(...)     // addr allows us to determine site (foo.isra.0)
    DW_AT_high_pc(...)
 DW_TAG_formal_parameter  // Get name, type info from abstract origin;
    DW_AT_abstract_origin // it points at abstract function parameter.
    DW_AT_location(...)   // Where is parameter stored (reg, const,
                          // stack?)
```

True signatures for GCC

- GCC can use abstract origin refs to convey when parameters are
  - Reordered (different ordering of abstract origin parameter references to original parameter order)
  - Missing (no reference or location specifying optimized-out parameter)
- We can get true signatures for these cases.
- Prototype https://github.com/alan-maguire/dwarves/tree/pahole-gcc-true
- Adds 704 .isra.0, .constprop.0 functions to BTF with their “.”-suffixed names

True signatures for GCC

- Example: parameters are reordered to reflect true signatures, e.g. sourcelevel
  - `static unsigned int __init add_to_rb(struct printk_ringbuffer *rb, struct printk_record *r);`

    becomes

  - `static unsigned int __init add_to_rb.constprop.0(struct printk_record *r, struct printk_ringbuffer *rb);`

Inline encoding

- A similar approach is used for inline sites
- We have site address (lowpc)
- We have reference to function and locations of (perhaps not all) parameters in locations
- Collate the info in BTF location representations where we tie together
  - Inlined function name (from abstract origin of concrete DW_TAG_inline_subroutine
  - Address (from DW_AT_low_pc)
  - Function signature (from abstract origin of each DW_TAG_formal_parameter)
  - Locations of each parameter (from locations in each DW_TAG_formal_parameter)

Inline encoding

x86_64 vmlinux built with gcc 14 has 443354 inline sites. Of these 443,354 locations:

- 318161 (~71%) have location information for all function parameters (where there are 0 or more parameters)
- 76520 (~17%) have incomplete location information; some parameters are available. For these the vast majority (67070) have only one missing parameter location.
- 48673 (~11%) have no location info for any of their parameters (where there are 1 or more parameters)

https://lore.kernel.org/bpf/20251008173512.731801-1-alan.maguire@oracle.com/

Inline tracing

With info about

-site

- what parameters are available and where

…we can use kprobe tracing to trace instances of inline sites

Need supplementary info for tracer to locate parameters (similar to what USDT tracing does in libbpf); e.g.

- Retrieve const, reg value or stack value for each parameter; and
- Report if parameter is missing
- Example implementation/test at https://lore.kernel.org/bpf/20251008173512.731801-16-alan.maguire@oracle.com/

Next Step

- Engage with dwarf working group about how to encode changed function signatures.
  - https://lists.dwarfstd.org/pipermail/dwarf-discuss/2025-December/002764.html
  - The above dwarf-discussion list is to ask dwarf community how to best encode changed signature in dwarf.
- Possible solutions:
  - Use DW_TAG_inlined_subroutine to encode true signature in LLVM/GCC
  - Dump true signatures to somewhere in ELF binary (not to pollute dwarf), e.g., “.BTF.true_signatures”.
  - Try to use locations in dwarf to get true signatures (and possibly llvm/gcc)
  - Directly generate related vmlinux BTF by compiler with true signatures, and then amended by additional things in resolve_btfids.

Next Step

- Engage with LLVM community for more code reviews. The llvm patch is off by default so less reviews…
- Engage with GCC community to encode changed function signatures in gcc. Define plan and timeline for gcc/llvm implementation.